

Chapter 20: Fibonacci Heaps

1) Binoomial heaps support in $O(\lg n)$ worst-case time the mergeable heap operations INSERT, MINIMUM, EXTRACT-MIN and UNION, plus the operations DECREASE-KEY and DELETE. Fibonacci heaps also support the same operations, but they have the advantage that the operations that do not involve deleting an element run in $O(1)$ amortized time.

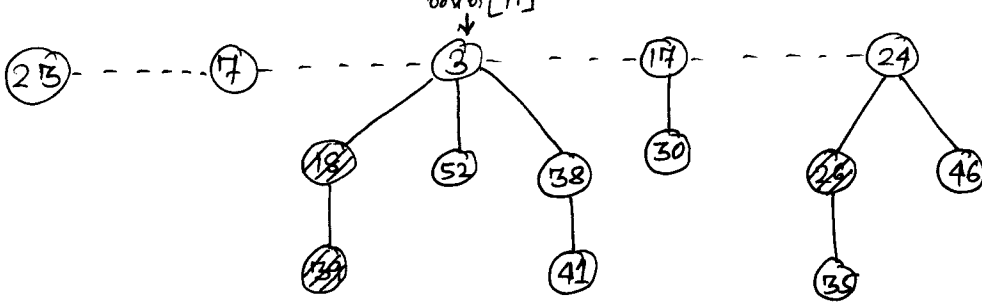
→ Fibonacci heaps are especially desirable when the number of EXTRACT-MIN and DELETE operations is small relative to other operations performed.

→ Like binoomial heaps, Fibonacci heaps are not designed to give efficient support to the operation SEARCH. Operations that refer to a given node therefore require a pointer to that node as part of their input. When we use a Fibonacci heap in an application, we often store a handle to the corresponding application object in each Fibonacci heap element, as well as a handle to corresponding Fibonacci heap element in each application object.

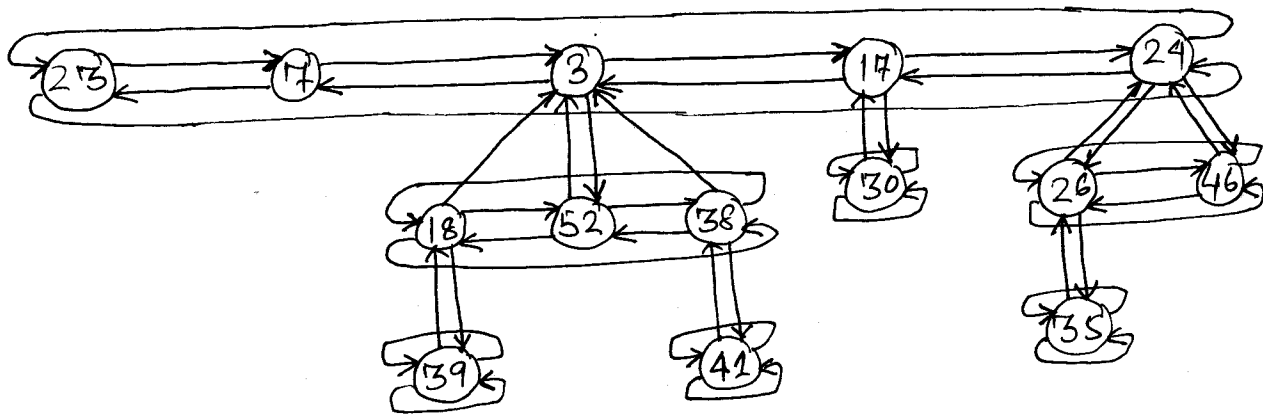
2) Structure of Fibonacci heaps:

A Fibonacci heap is a collection of min-heap-ordered trees. The trees in a Fibonacci heap are not constrained to be binoomial trees. Another dissimilarity between Fibonacci heaps and Binoomial heaps is that the trees in a Fibonacci heap are unordered (but rooted).

For example:-



A Fibonacci heap with 5 non-heap-ordered trees and 14 nodes.
The marked nodes have been crossed.



Every node x in a ~~bivariate~~ ^{Fibonacci} heap maintains the following fields:-

- (i) $\text{parent}[x]$: a pointer to the parent of x
- (ii) $\text{child}[x]$: a pointer to one of x 's children. The children of x are linked together in a circular, doubly linked list, which we call the "child list" of x . The order in which the siblings appear in a child list is arbitrary.
- (iii) $\text{left}[x]$: a pointer to x 's left sibling.
- (iv) $\text{right}[x]$: a pointer to x 's right sibling. If x is the only node in its circular doubly linked list then $\text{left}[x] = \text{right}[x] = x$.
- (v) $\text{degree}[x]$: the no. of children x has.

v) $\text{mark}[x]$: It is a boolean valued field. A node is marked if at least one of its children was cut since this node was made a child of another node. All roots are unmarked. Newly created nodes are unmarked. And a node x becomes unmarked whenever it is made the child of another node.

→ Circular, doubly linked lists have two advantages for use in Fibonacci heaps.

(i) We can remove a node from a circular, doubly linked list in $O(1)$ time.

(ii) Given two such lists, we can concatenate them into one circular, doubly linked list in $O(1)$ time.

→ A Fibonacci heap object H contains the following fields:-

(i) $\text{min}[H]$: points to that root which contains the minimum key. If Fibonacci heap H is empty, $\text{min}[H] = \text{NIL}$.

All the roots in a Fibonacci heap are linked together into a circular, doubly linked list called the "root list". The order of the trees in a root list is arbitrary.

(ii) $n[H]$: holds the no. of nodes in H .

(iii) $t[H]$: the no. of trees in the root list of H .

(iv) $m[H]$: the no. of marked nodes in H .

3) Fibonacci heaps have a more relaxed structure than binomial heaps, allowing for improved asymptotic time bounds. Work that maintains the structure can be delayed until it is convenient to perform.

Fibonacci heaps are a good example of a data structure designed with amortized analysis in mind. We shall use the potential method to analyze the performance of Fibonacci heap operations.

Potential function: The potential of Fibonacci heap H is defined by

$$\phi(H) = t(H) + 2m(H)$$

The potential of a set of Fibonacci heaps is the sum of the potentials of its constituent Fibonacci heaps. We assume that a Fibonacci heap application begins with no heaps, so that the initial potential is 0. The potential is non-negative at all subsequent times. The total amortized cost for a sequence of operations forms an upper bound on the total actual cost for the sequence of operations.

We assume that a unit of potential is sufficient to pay for the costliest constant amount of work.

Maximum degree: The amortized analysis assumes that $D(n)$ forms an upper bound on the maximum degree of any node in an n -node Fibonacci heap. I shall be shown that $D(n) = O(\lg n)$.

4) Mergable-heap operations: If only the operations - MAKE-HEAP, INSERT, MINIMUM, EXTRACT-MIN and UNION - are to be supported, each Fibonacci heap is simply a collection of unordered binomial trees.

An "unordered Binomial tree" is like a binomial tree except for the fact that there is no ordering among the children of a node. The unordered binomial tree U_0 consists of a single node. The unordered binomial tree U_k consists of two unordered binomial trees U_{k-1} such that one U_{k-1} is made a child of the

Other U_{k-1} . The 1st 3 properties of binomial trees hold for unordered binomial trees as well. But the 4th property says:-

4) For the unordered binomial tree U_k , the root has degree k , which is greater than that of any other node. The children of the root are roots of subtrees $U_0, U_1, U_2, \dots, U_{k-1}$ but in any order.

→ If an n -node Fibonacci heap is a collection of unordered binomial trees, then $D(n) = \lg n$.

→ The key idea in the mergeable heap operations on Fibonacci heap is to delay work as long as possible. If the no. of trees in a Fibonacci heap is small, then during an EXTRACT-MIN operation we can quickly determine which of the remaining nodes becomes the new minimum node.

But we pay a price for ensuring that the no. of trees is small: it can take up to $\Omega(\lg n)$ time to insert a node into a binomial heap or write two "binomial heaps". We do not attempt to consolidate trees in a Fibonacci heap when we insert a new node or write two heaps. We save the consolidation part for the EXTRACT-MIN operation which is when we really need to find the new minimum node.

5) Creating a new Fibonacci heap:

The MAKE-FIB-HEAP procedure makes an empty Fibonacci heap. It allocates and returns the Fibonacci heap object H , where $n[H] = 0$ and $\text{min}[H] = \text{NIL}$. [May be you do not need to maintain the fields $t[H]$ and $m[H]$ in the object H , since they are not concerned with maintenance of the

structure of a Fibonacci heap.]

Because there isn't any tree in H , $e[H] = 0$ & $m[H] = 0$ implying that the potential of an empty Fibonacci heap is $\phi[H] = 0$. Thus the amortized cost of the MAKE-FIB-HEAP procedure is equal to its $O(1)$ actual cost.

6) Inserting a node:

The following procedure inserts node x into Fibonacci heap H :-

FIB-HEAP-INSERT (H, x)

- 1 $\text{degree}[x] \leftarrow 0$
- 2 $\text{parent}[x] \leftarrow \text{NIL}$
- 3 $\text{child}[x] \leftarrow \text{NIL}$
- 4 $\text{left}[x] \leftarrow \text{NIL}$
- 5 $\text{right}[x] \leftarrow \text{NIL}$
- 6 $\text{mark}[x] \leftarrow \text{FALSE}$
- 7 concatenate the root list containing x with the root list containing H
- 8 If $\text{min}[H] = \text{NIL}$ or $\text{Key}[x] < \text{Key}[\text{min}[H]]$
- 9 then $\text{min}[H] \leftarrow x$
- 10 $m[H] \leftarrow m[H] + 1$

→ Like the BINOMIAL-HEAP-INSERT procedure, FIB-HEAP-INSERT make the new node into a single-node min-heap ordered tree and then merges this new FIB heap with the old FIB heap. But unlike the BINOMIAL-HEAP-INSERT procedure, it makes no attempt to consolidate the trees with the Fibonacci heap. Thus, if K consecutive FIB-HEAP-INSERT operations occurs, then K single-node trees are added to the root list.

→ If H be the 1/P Fibonacci heap and H' be the resulting Fibonacci heap. Then, $t(H') = t(H) + 1$ and $m(H') = m(H)$.

The increase in potential is :-

$$\begin{aligned}\phi(H') - \phi(H) &= (t(H) + 1) + 2m(H) - (t(H) + 2m(H)) \\ &= 1\end{aligned}$$

$$\begin{aligned}\text{amortized cost} &= \text{actual cost} + \text{increase in potential} \\ &= O(1) + 1 = O(1).\end{aligned}$$

7) Finding the minimum node:

The minimum node of a Fibonacci heap is pointed to by $\text{min}[H]$, so we can find the minimum node in $O(1)$ actual time. Because the potential of H does not change, the amortized cost of this operation equals its $O(1)$ actual cost.

8) Uniting two Fibonacci heaps:

The procedure `FIB-HEAP-UNION` unites 2 Fibonacci heaps H_1 and H_2 , destroying them in the process.

`FIB-HEAP-UNION ( $H_1, H_2$ )`

- 1 $H \leftarrow \text{MAKE-FIB-HEAP}()$
- 2 $\text{min}[H] \leftarrow \text{min}[H_1]$
- 3 concatenate the root list of H_2 with the root list of H
- 4 if $(\text{min}[H_1] = \text{NIL})$ or $(\text{min}[H_2] \neq \text{NIL}$ and $\text{min}[H_2] < \text{min}[H_1])$
- 5 then $\text{min}[H] \leftarrow \text{min}[H_2]$
- 6 $m[H] \leftarrow m[H_1] + m[H_2]$

7 free the objects $H1$ and $H2$

8 return H

→ The **FIB-HEAP-UNION** procedure simply concatenates the root lists of $H1$ and $H2$ without performing any consolidation of trees.

The change in potential is 0, since neither the no. of trees increase nor the no. of marked nodes. Therefore, the amortized cost of **FIB-HEAP-UNION** equals its $O(1)$ actual cost.

9) Extracting the minimum node:

This is where the delayed work of consolidating trees in the root list finally occurs.

FIB-HEAP-EXTRACT-MIN works by first making a root out of each of the minimum node's children and removing the minimum node from the root list. It then consolidates the root list by linking roots of equal degree until at most one root remains of each degree.

FIB-HEAP-EXTRACT-MIN(H)

```
1 min_node ← min[H]
2 if min_node ≠ NIL
3   then for each child  $x$  of min_node
4     do add  $x$  to the root list of  $H$ 
5     parent [ $x$ ] ← NIL
6   remove min_node from the root list of  $H$ 
7   if min_node = right[min_node] ▷ was it the only
8     then min[H] ← NIL           ▷ node in the heap?
9   else min[H] ← right[min_node]
10  CONSOLIDATE( $H$ )
```


11 $n[H] \leftarrow n[H] - 1$

12 return z

→ When you convert the children of min-node into roots, you add them to the left of min-node in the root list.

When we remove min-node from the root list, we set $\text{min}[H]$ to point to $\text{right}[\text{min-node}]$ which may not necessarily be the new minimum node. The auxiliary procedure CONSOLIDATE takes care of this situation.

→ Consolidating the root list consists of repeatedly executing the following steps until every root in the root list has a distinct degree value:-

- 1) Find two roots x and y in the root list with the same degree, where $\text{Key}[x] \leq \text{Key}[y]$
- 2) Link y to x : remove y from the root list and make y a child of x . Increment the degree of x , and if there is any mark on y , clear it. This is performed by the FIB-HEAP-LINK procedure.

→ The procedure CONSOLIDATE uses an auxiliary array $A[0 \dots D(n[H])]$. If $A[i] = y$, then y is a root with degree equal to i , i.e. $\text{degree}[y] = i$.

CONSOLIDATE(H)

```
1  for  $i \leftarrow 0$  to  $D(\min[H])$ 
2      do  $A[i] \leftarrow \text{NIL}$ 
3  for each node  $w$  in the root list of  $H$ 
4      do  $x \leftarrow w$ 
5           $d \leftarrow \text{degree}[x]$ 
7          while  $A[d] \neq \text{NIL}$ 
8              do  $y \leftarrow A[d]$   $\triangleright$  Another node with same
                               degree as  $x$ 
9                  if  $\text{Key}[x] > \text{Key}[y]$ 
10                     then exchange  $x \leftrightarrow y$ 
11                     FIB-HEAP-LINK( $H, y, x$ )
12                      $A[d] \leftarrow \text{NIL}$ 
13                      $d \leftarrow d + 1$ 
14              $A[d] \leftarrow x$ 
15   $\min[H] \leftarrow \text{NIL}$ 
16  for  $i \leftarrow 0$  to  $D(\min[H])$ 
17      do if  $A[i] \neq \text{NIL}$ 
18          then add  $A[i]$  to the root list of  $H$ 
19              if  $\min[H] = \text{NIL}$  or  $\text{Key}[A[i]] < \text{Key}[\min[H]]$ 
20                  then  $\min[H] \leftarrow A[i]$ 
```

FIB-HEAP-LINK(H, y, x)

```
1  remove  $y$  from the root list of  $H$ 
2  make  $y$  a child of  $x$  and increment  $\text{degree}[x]$ 
3   $\text{mark}[y] \leftarrow \text{FALSE}$ 
```

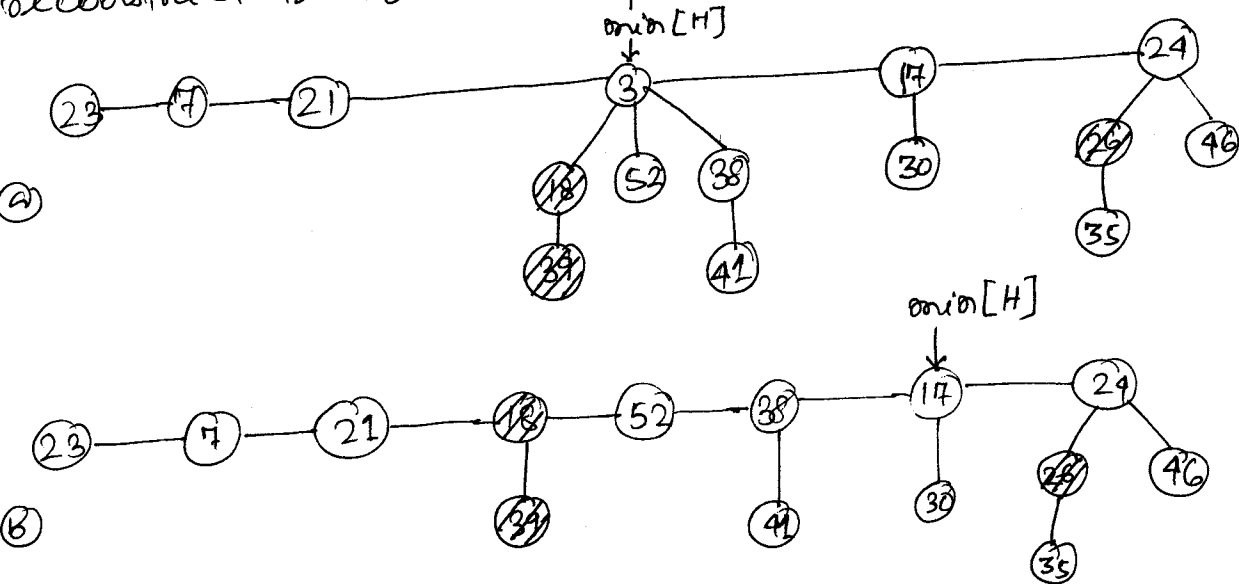
→ The purpose of the while loop is to repeatedly link the root x of the tree containing node w to another tree whose root has the same degree as x , until no other root has the same degree.

The while loop maintains the following invariant:-

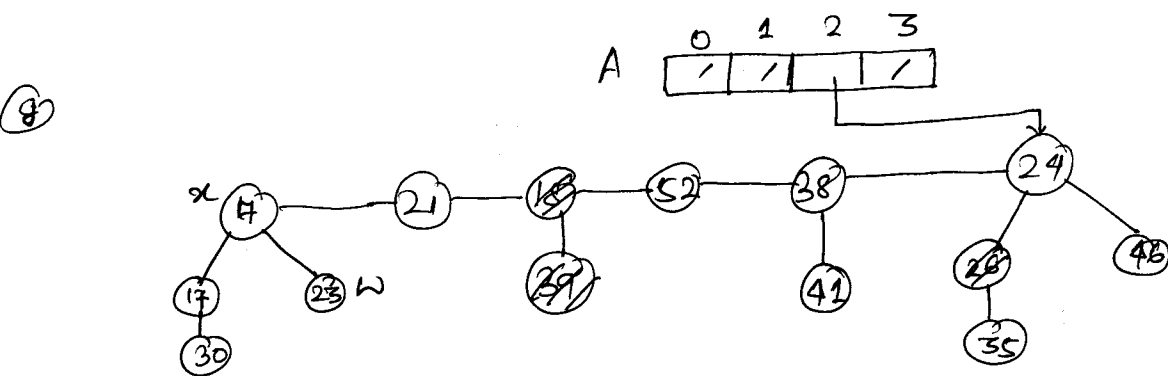
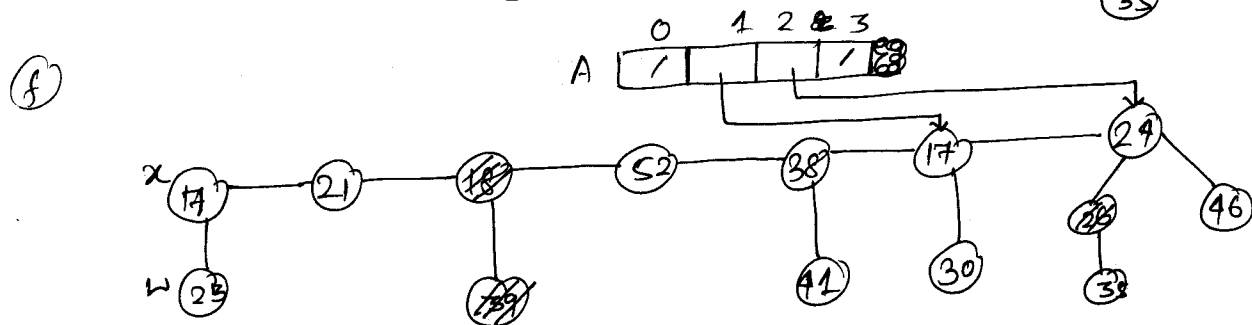
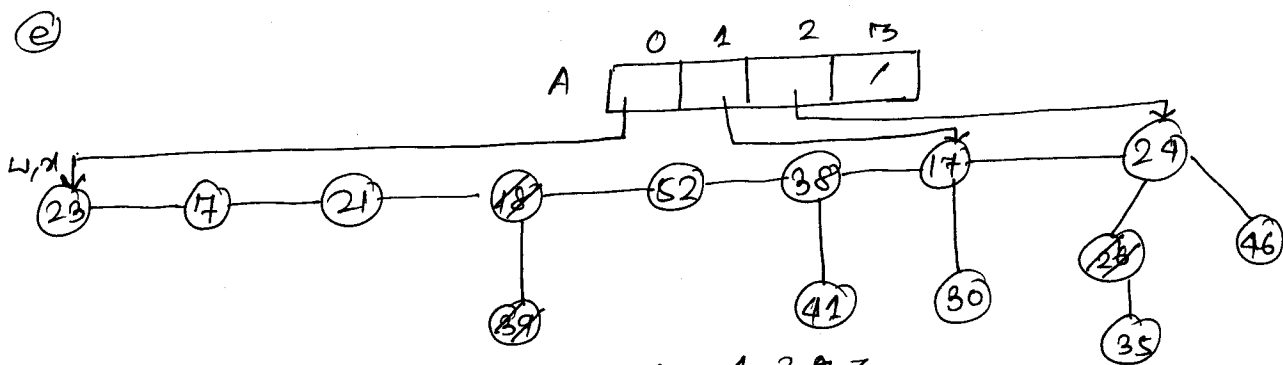
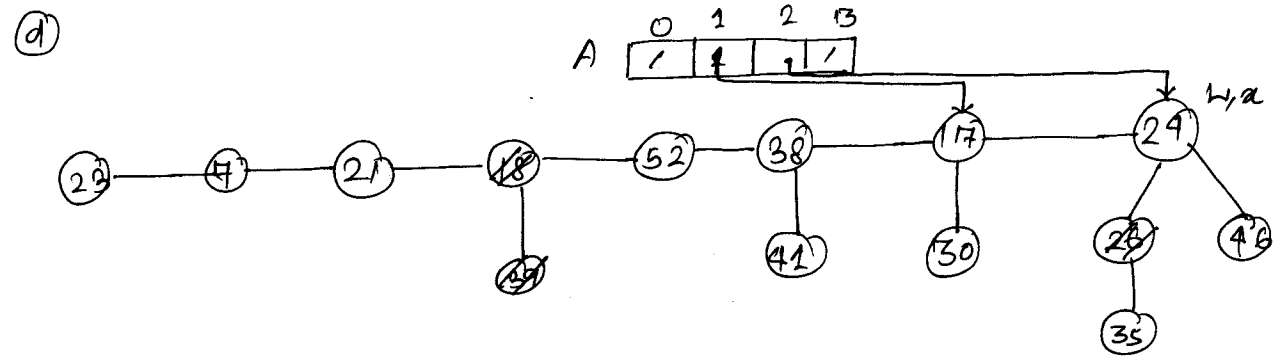
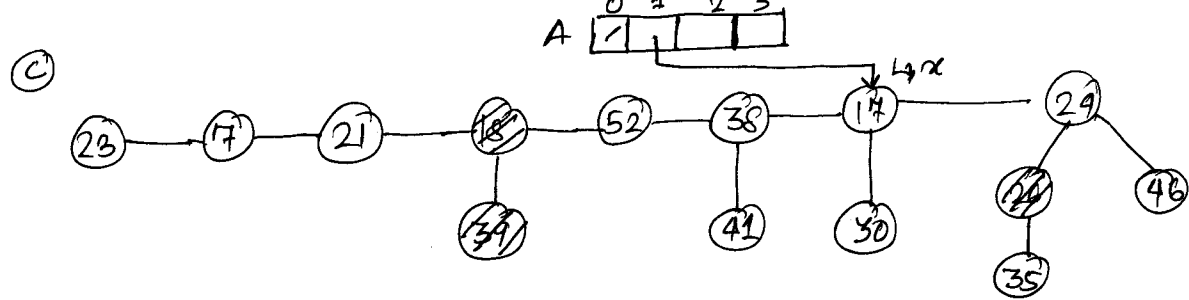
At the start of each iteration of the while loop $d = \text{degree}[x]$.

At the end of each iteration of the for loop, x points to a root with unique degree. We set the entry $A[\text{degree}[x]]$ to point to x . Hence, when the for loop terminates, there remain at max one root of each degree and the array A points to each remaining root.

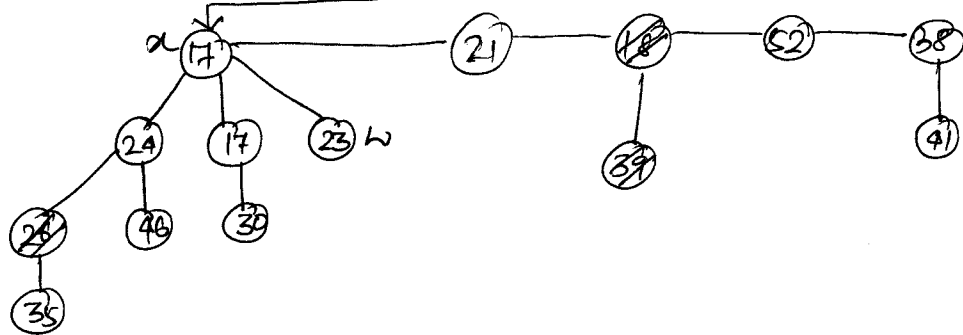
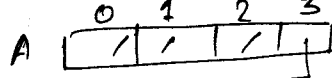
All that remains is to perform cleanup. Line 15 empties the root list and the next few lines reconstruct it from array A .



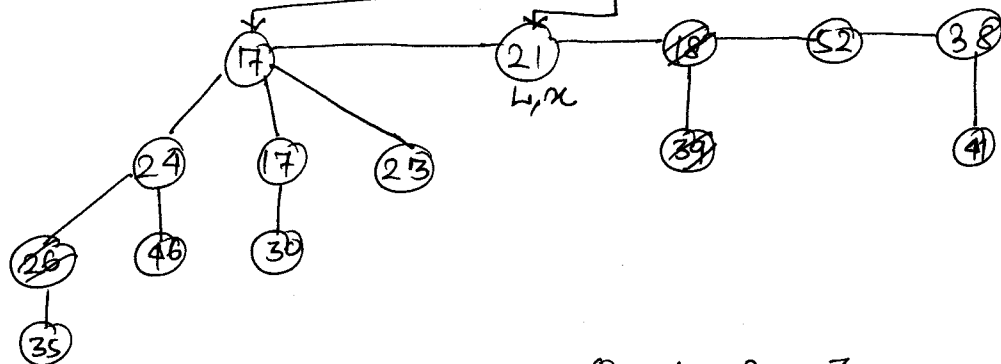
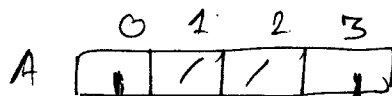
Suppose we traverse the root list of H by following the right pointers from $\text{union}[H]$.



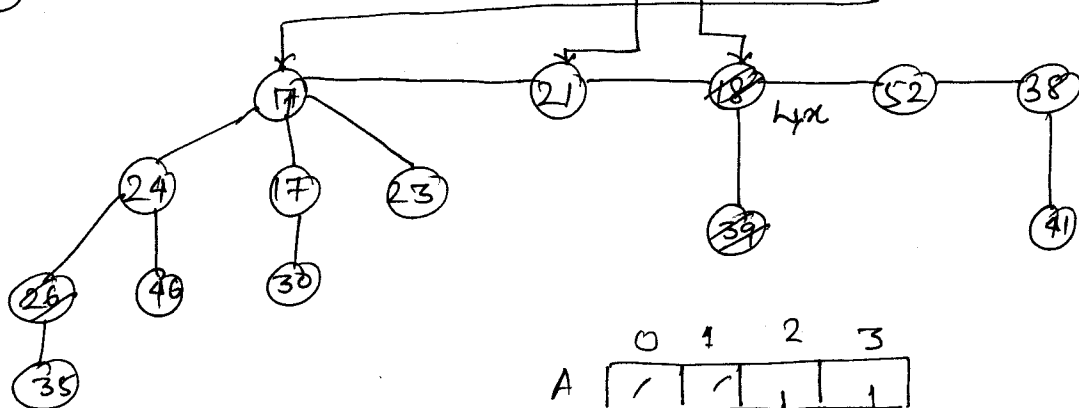
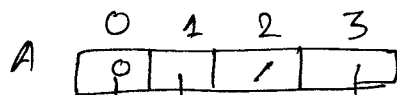
h



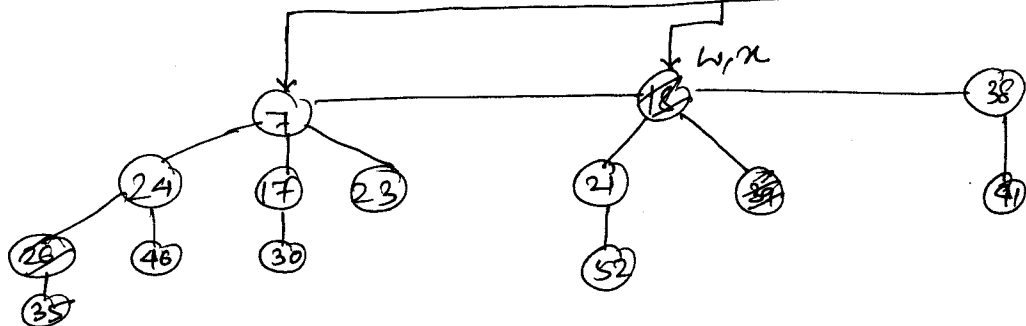
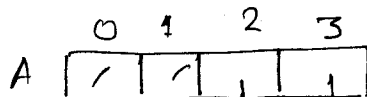
i

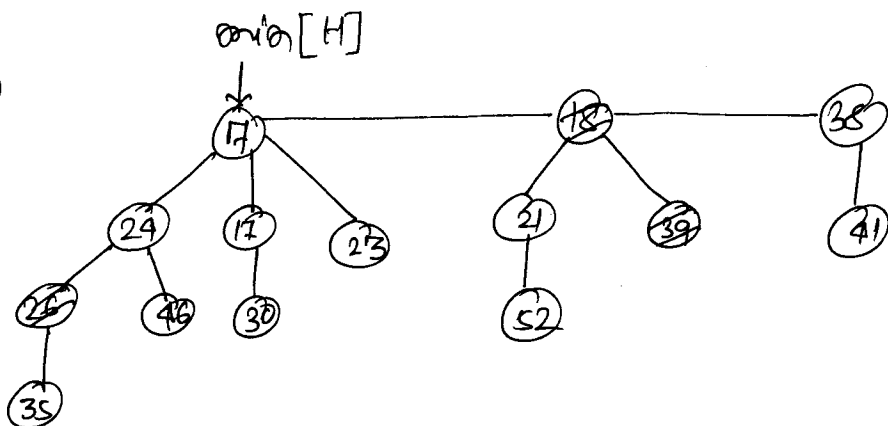
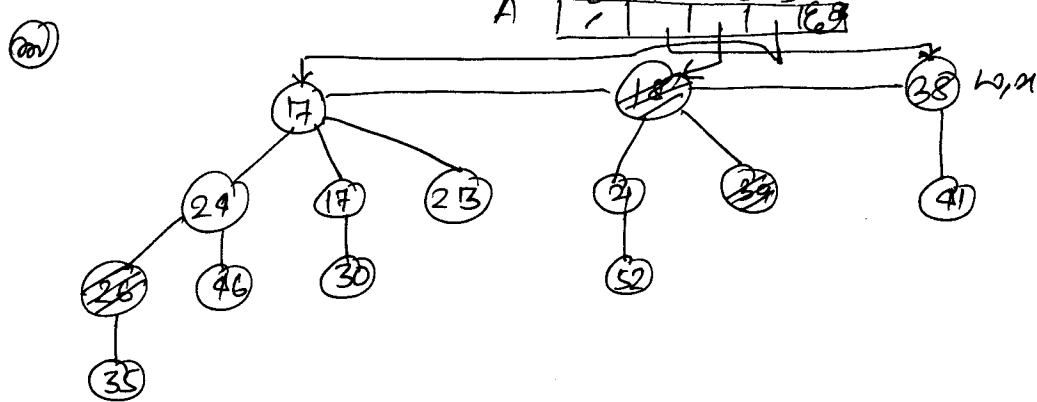


j



k+2





→ If all trees in the Fibonacci heap are unordered binomial trees before FIB-HEAP-EXTRACT-MIN is executed, then they are all unordered binomial trees afterward. There are 2 ways in which trees are changed. First, When each child x of the root min-node becomes a root. But each such child is itself an unordered binomial tree [By the 4th property]. Second, when 2 trees T_k are linked by FIB-HEAP-LINK. The resulting tree T_{k+1} is also an unordered binomial tree.

→ The sum times of the for loops of lines 12 and 15-20 of CONSOLIDATE is $O(D(m))$. The size of the root list upon calling CONSOLIDATE is $t(H) + D(m) - 1$. Hence, the sum times of the for loop of lines 3-14 is $O(t(H) + D(m))$. Hence the total actual work done by FIB-HEAP-EXTRACT-MIN is $O(D(m)) + O(t(H) + D(m)) = O(t(H) + D(m)) \cdot 14$

Potential before extracting the minimum node is $t(H) + 2m(H)$. Max. no. of roots at the end $= D(n) + 1 = \text{size of array } A$. Hence potential afterward $= D(n) + 1 + 2m(H)$ [there is no change in no. of marked ~~roots~~ nodes].

$\therefore$ Amortized cost = actual cost + change in potential
 $= O(D(n) + t(H)) + D(n) + 1 + 2m(H) - (t(H) + 2m(H))$
 $= O(D(n))$

[We can scale up the units of potential to dominate the constant hidden in $t(H)$].

10) Decreasing a Key:

FIB-HEAP-DECREASE-KEY(H, x, K)

```

1  if  $K > \text{Key}[x]$ 
2      then error "new Key is greater than current Key"
3   $\text{Key}[x] \leftarrow K$ 
4   $y \leftarrow \text{parent}[x]$ 
5  if  $y \neq \text{NIL}$  and  $\text{Key}[x] < \text{Key}[y]$ 
6      then CUT( $H, x, y$ )
7          CASCADING-CUT( $H, y$ )
8  if  $\text{Key}[x] < \text{Key}[\text{min}[H]]$ 
9      then  $\text{min}[H] \leftarrow x$ 

```

$\triangleright$ only Key of x gets affected. check whether
 $\triangleright$ it has been made lesser than $\text{Key}[\text{min}[H]]$.

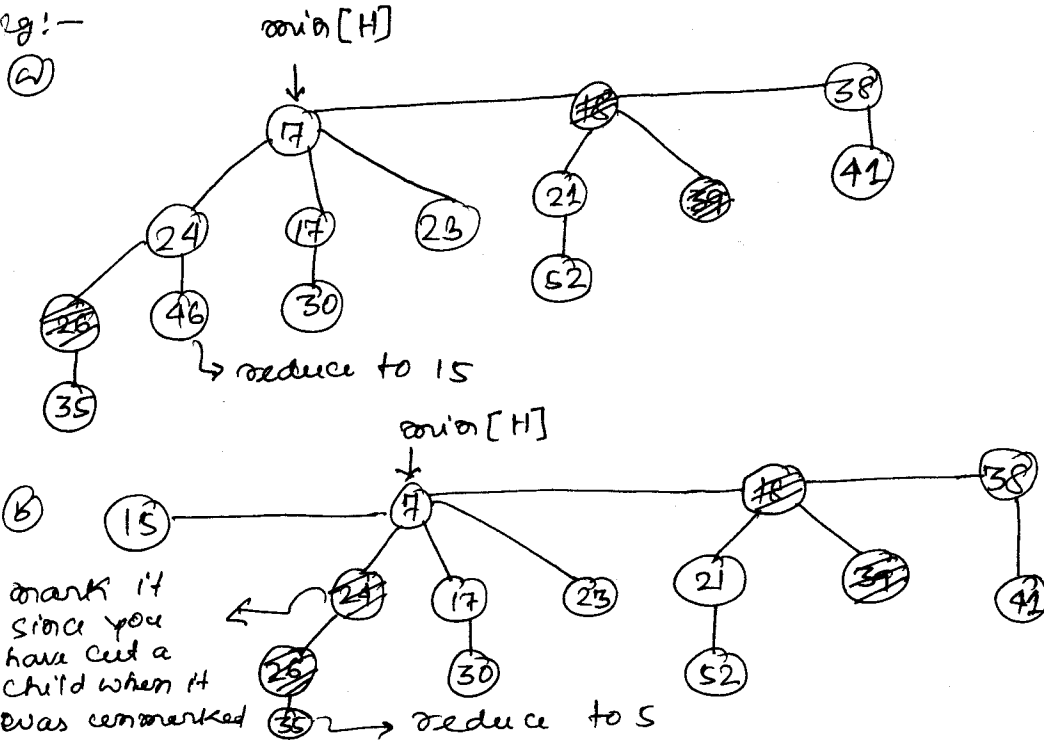
CUT(H, x, y)

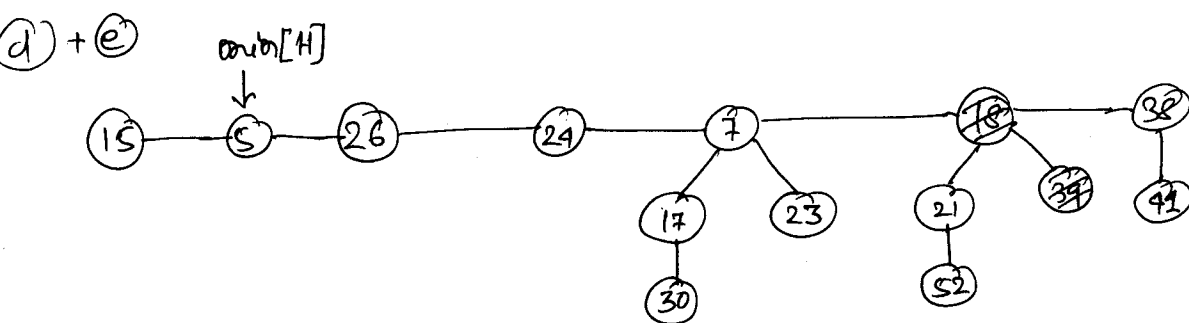
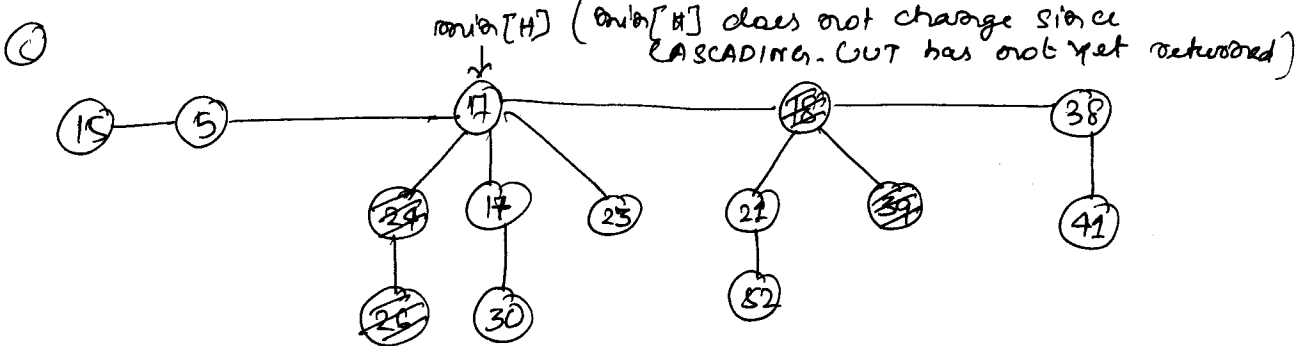
- 1 remove x from the child list of y , decrementing $\text{degree}[y]$
- 2 add x to the root list of H
- 3 $\text{parent}[x] \leftarrow \text{NIL}$
- 4 $\text{mark}[x] \leftarrow \text{FALSE}$

CASCADING-CUT(H, y)

- 1 $z \leftarrow \text{parent}[y]$
- 2 if $z \neq \text{NIL}$
- 3 then if $\text{mark}[y] = \text{FALSE}$
- 4 then $\text{mark}[y] \leftarrow \text{TRUE}$
- 5 else $\text{CUT}(H, y, z)$
- 6 $\text{CASCADING-CUT}(H, z)$

What is the reason behind marking the nodes and performing CASCADING cuts ???





→ The amortized cost of FIB-HEAP-DECREASE-KEY is only $O(1)$: The FIB-HEAP-DECREASE-KEY procedure takes $O(1)$ time plus the time to perform cascading cuts. Suppose that from a given invocation of FIB-HEAP-DECREASE-KEY, you have c times recursive invocation of CASCADING-CUT. Each call of CASCADING-CUT takes $O(1)$ time exclusive of recursive calls. Thus, the "actual cost" of FIB-HEAP-DECREASE-KEY including all recursive calls is $O(c)$.

To compute the change in potential, suppose h represents the Fibonacci heap prior to the FIB-HEAP-DECREASE-KEY operation. Every recursive call of CASCADING-CUT, except for the last one, cuts a marked node and clears the mark bit. Hence, the no. of trees afterward = $t(H) + c = t(H)$ original trees + $c - 1$ produced by cascading cuts + tree rooted at x .
 No. of marked nodes = max. of $m(H) - c + 2 (= c - 1$ were unmarked by cascading cuts and the last call

of CASCADING-CUT may have marked a node).

$\therefore$ change in potential is at max. =

$$t(H) + c + 2(m(H) - c + 2) - (t(H) + 2m(H)) \\ = 4 - c$$

amortized cost = actual cost + change in potential

$$= O(c) + 4 - c = O(1)$$

[Since we can scale up the units of potential to dominate the constant hidden in $O(c)$.]

→ It now becomes clear, as to why the potential function was defined to include a term that is twice the no. of marked nodes. When a marked node is cut by a cascading cut, its mark bit is cleared, so that the decrease in potential is of 2. One unit of potential pays for the cut and the clearing of the mark bit, while the other unit compensates for the unit increase in potential due to node y becoming a root.

11) Deleting a node :-

FIB-HEAP-DELETE(H, x)

- 1 FIB-HEAP-DECREASE-KEY($H, x, -\infty$)
- 2 FIB-HEAP-EXTRACT-MIN(H)

We assume that there is no Key value of $-\infty$ currently in the Fibonacci heap.

Amortized time of FIB-HEAP-DELETE = $O(1)$

amortized time of FIB-HEAP-DECREASE-KEY

+

$O(D(m))$ amortized time of FIB-HEAP-EXTRACT-MIN
 $= O(D(m))$

→ The operations of decreasing a key and deleting a node do not preserve the property that all trees in a Fibonacci heap are considered binomial trees. [FIB-HEAP-DECREASE-KEY is the culprit. If the degree of a root is K it should have a total of 2^K nodes for it to be a binomial tree. But when you perform a cut you simply remove that child and add it to the root list, so that the no. of nodes in that tree is now $< 2^K$].